

B1B13PPS

Průmyslové počítačové systémy

Michal Brejcha

`brejcmic@fel.cvut.cz`

ČVUT v Praze, FEL

Praha, 2025

Obsah

- 1 Sčítání a odčítání v jiných soustavách
- 2 Vyjádření znaménka - záporná čísla
- 3 Násobení a dělení
- 4 Plovoucí řádová čárka

Téma

- 1 Sčítání a odčítání v jiných soustavách
- 2 Vyjádření znaménka - záporná čísla
- 3 Násobení a dělení
- 4 Plovoucí řádová čárka

Sčítání

Sčítání v soustavě o základu 10

1. člen X	1	2	2
2. člen Y	2	9	1
přenos	1		
výsledek	4	1	3

- Přenos při $x + y > 10$
- Zapisujeme $x + y - 10$

Sčítání v soustavě o základu 8

1. člen X	1	7	2
2. člen Y	4	4	3
přenos	1		
výsledek	6	3	5

- Přenos při $x + y > 8$
- Zapisujeme $x + y - 8$

Sčítání vlastnosti

- Ve všech soustavách stejný postup.
- Při sčítání dvou sčítanců může být přenos maximálně 1.
- Při sčítání dvou sčítanců může být výsledek o jednu pozici širší než jsou oba sčítance.

Později bude dávat smysl rozšíření obou sčítanců nulami vlevo na možnou délku výsledku.

Příklad pro binární hodnoty:

1. člen X	0	1	1	1	0	1	0
2. člen Y	0	1	0	1	0	1	1
přenos	1	1	1		1		
výsledek	1	1	0	0	1	0	1

Odčítání

Odčítání v soustavě o základu 10 Odčítání v soustavě o základu 8

1. člen X		2	9	1
2. člen Y	-	1	2	2
výpůjčka			-1	
výsledek		1	6	9

1. člen X		4	4	3
2. člen Y	-	1	7	2
výpůjčka			-1	
výsledek		2	5	1

- Přenos při $x + y > 10$
- Zapisujeme $x - y + 10$

- Výpůjčka při $x - y < 0$
- Zapisujeme $x - y + 8$

Odčítání vlastnosti

- Ve všech soustavách stejný postup.
- Při rozdílu dvou členů může být výpůjčka maximálně 1.
- Při přímém vyjádření znaménka musí být menšenec větší než menšitel.
- Pro větší menšitel než menšenec stále odečítáme menší číslo od většího a měníme znaménko u výsledku.
- Výsledek může být jen kratší, viz příklad.

Příklad pro binární hodnoty:

1. člen X		1	1	1	0	1	0
2. člen Y	-	1	0	1	0	1	1
výpůjčka			-1	-1	-1	-1	
výsledek		0	0	0	1	1	1

Téma

1 Sčítání a odčítání v jiných soustavách

2 Vyjádření znaménka - záporná čísla

3 Násobení a dělení

4 Plovoucí řádová čárka

Zobrazení záporných čísel

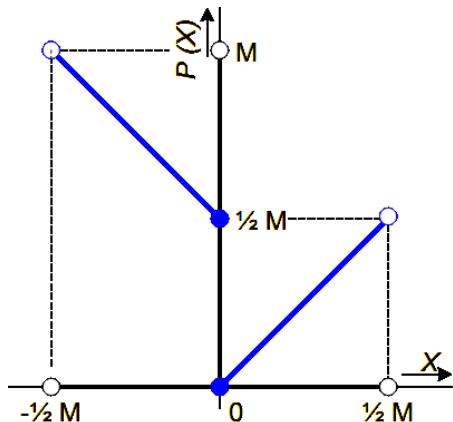
- Polyadické soustavy obsahují pouze nezáporná čísla.
- Hledáme transformaci mezi omezenou množinou celých čísel a omezenou množinou čísel nezáporných.
- Očekáváme binární reprezentaci pro HW aplikace - zápis do registru v mikroprocesoru.
- Očekáváme snadný algoritmus provádění aritmetických operací - ALU v mikroprocesoru.
- Nejpoužívanější číselné kódy
 - přímý,
 - aditivní,
 - doplňkový.

Přímý kód

- Volba znaménka dle MSB
+ odpovídá 0
- odpovídá 1
- Zbývající bity reprezentují absolutní hodnotu
- Reprezentace je přirozená člověku
- Nepraktické rozšiřování na více bitů
- Příklad:

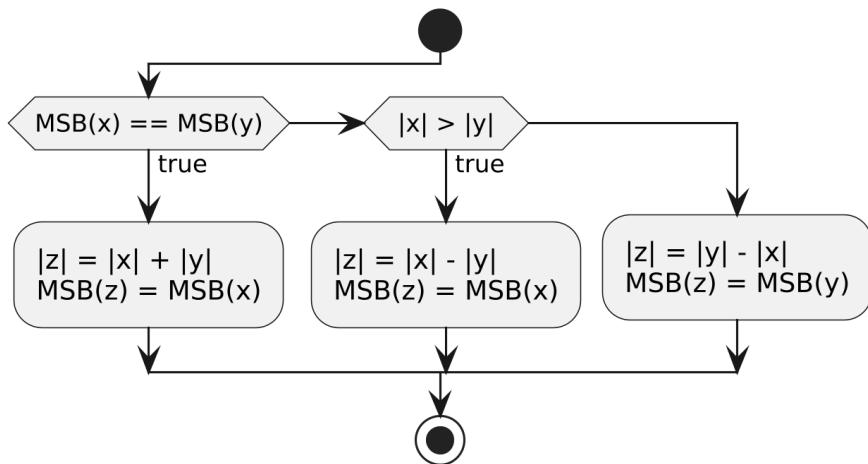
$$(6)_{10} = (00000110)_2$$

$$(-6)_{10} = (10000110)_2$$



$$P(X) = \begin{cases} X & \text{pro } X > 0 \\ |X| + \frac{1}{2}M & \text{pro } X \leq 0 \end{cases}$$

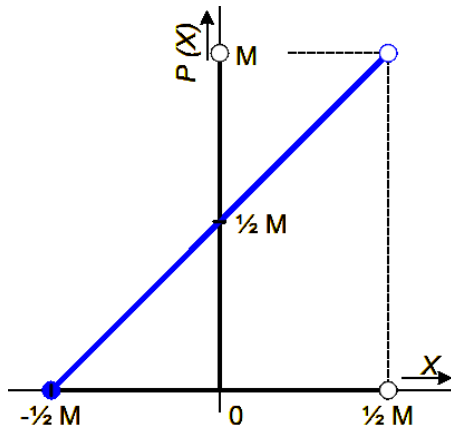
Přímý kód - sčítání



- Nesmí dojít k přetečení při součtu absolutních hodnot.

Aditivní kód

- Kód s posunutou nulou o K
- Zachovává relace $>$, $<$, $=$
- Jednoduchý převod
- Speciálně pro $K = M/2$ lze negací MSB získat totéž číslo v doplňkovém kódu
- Obtížné rozšiřování na více bitů



$$P(X) = X + K$$

$$X \geq -K, X \leq M - K$$

Aditivní kód - sčítání

Řešíme součet pro: $X = x + A$, $Y = y + A$

- A ... je posun,
- x, y ... původní členy součtu
- X, Y ... členy součtu po převodu

Po dosazení: $X + Y = x + A + y + A = (x + y) + 2A$

- Správný výsledek pokud na konci operace odečteme jeden posun
- **Příklad:** $x = 11$, $y = -6$, $A = 32$

$$X = (101011)_2$$

$$Y = (011010)_2 = (26)_{10}$$

$$X + Y = (1000101)_2$$

$$-A = -(100000)_2$$

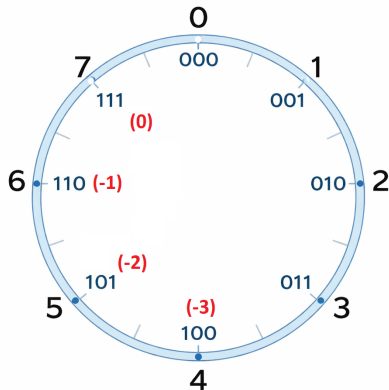
$$Z = (100101)_2 = (37)_{10}$$

Doplňkový kód - 1. doplněk

- Doplněk do modulu mřížky M
- Problém s přítomností dvou nul při přetečení
- Snadný převod mezi kladnou a zápornou hodnotou:

$(X)_{10}$	$(X)_2$	$(-X)_{10}$	$(-X)_2$
0	000	-0	111
1	001	-1	110
2	010	-2	101
3	011	-3	100

Jen inverze všech bitů.



Doplňkový kód - 1. doplněk - výpočty

$$1 + 2 = [1] + [2] = [3] = 3 \text{ OK}$$

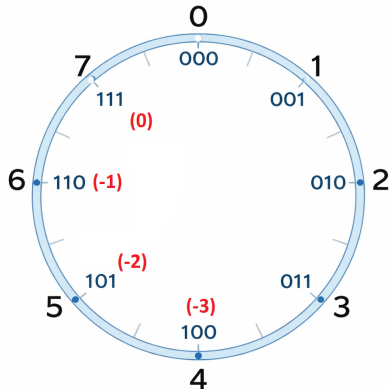
$$1 - 2 = [1] + [5] = [6] = -1 \text{ OK}$$

$$-1 + 2 = [6] + [2] = [8] = 0 \text{ X}$$

$$-1 - 2 = [6] + [5] = [11] = 3 \text{ X}$$

Problém s přetékaním:

- při přetečení je nutné přičíst jedničku
- do operace lze přímo zavést bit přetečení



Sčítání - 1. doplněk

Řešíme: $(9)_{10} - (15)_{10} = (-6)_{10}$

Převody:

$$(9)_{10} = (001001)_2$$

$$(15)_{10} = (001111)_2$$

$$(-15)_{10} = (110000)_2$$

	0	0	1	0	0	1
	1	1	0	0	0	0
0	1	1	1	0	0	1
						+0
	1	1	1	0	0	1

Řešíme: $(15)_{10} - (9)_{10} = (6)_{10}$

Převody:

$$(15)_{10} = (001111)_2$$

$$(9)_{10} = (001001)_2$$

$$(-9)_{10} = (110110)_2$$

	0	0	1	1	1	1
	1	1	0	1	1	0
1	0	0	0	1	0	1
						+1
	0	0	0	1	1	0

Každý převod musí obsahovat další bit navíc (dohromady 2) kvůli

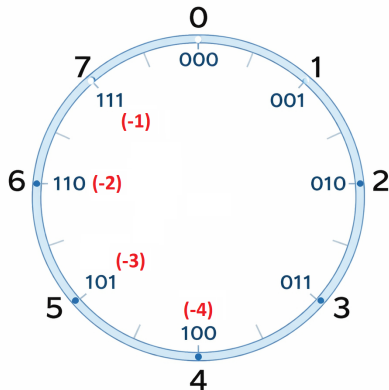
- existenci přetečení při součtu kladných čísel,
- existenci nové informace o znaménku.

Doplňkový kód - 2. doplněk

- Doplněk do maxima mřížky
 $M - 1$
- Jen jedna nula
- Relativně snadný převod mezi kladnou a zápornou hodnotou:

$(X)_{10}$	$(X)_2$	$(-X)_{10}$	$(-X)_2$
0	000	-0	/
1	001	-1	111
2	010	-2	110
3	011	-3	101
4	/	-4	100

Inverze všech bitů + 1.



Sčítání - 2. doplněk

Řešíme: $(9)_{10} - (15)_{10} = (-6)_{10}$
Převody:

$$\begin{aligned}(9)_{10} &= (001001)_2 \\ (15)_{10} &= (001111)_2 \\ (-15)_{10} &= (110001)_2\end{aligned}$$

← 0		0	0	1	0	0	1
← 1		1	1	0	0	0	1
← 1		1	1	1	0	1	0

Řešíme: $(15)_{10} - (9)_{10} = (6)_{10}$
Převody:

$$\begin{aligned}(15)_{10} &= (001111)_2 \\ (9)_{10} &= (001001)_2 \\ (-9)_{10} &= (110111)_2\end{aligned}$$

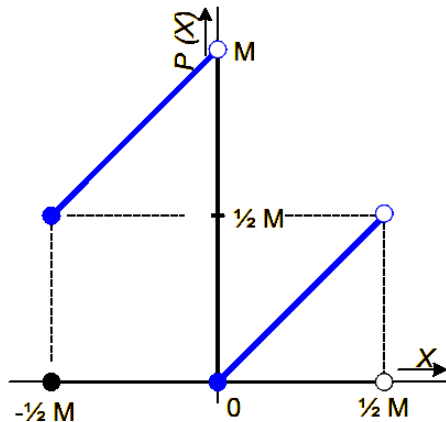
← 0		0	0	1	1	1	1
← 1		1	1	0	1	1	1
← 0		0	0	0	1	1	0

Totéž co bylo u prvního doplňku, musím přidat nejméně 2 bity kvůli:

- existenci přetečení při součtu,
- existenci nové informace o znaménku.

Doplňkový kód

- Znaménko je patrné z MSB
- Snadná HW implementace a výpočty
- Snadné rozšiřování počtu bitů
 - $X \geq 0$ rozšiřuje 0 vlevo
 - $X < 0$ rozšiřuje 1 vlevo
- Snadné násobení



$$P(X) = \begin{cases} X & \text{pro } X \geq 0 \\ X + M & \text{pro } X < 0 \end{cases}$$

Téma

1 Sčítání a odčítání v jiných soustavách

2 Vyjádření znaménka - záporná čísla

3 Násobení a dělení

4 Plovoucí řádová čárka

Násobení ve 2. doplňku

- Násobení v ostatních soustavách funguje stejně jako v desítkové soustavě - násobení pod sebou,
- je extrémně jednoduché pro binární soustavu, protože opisujeme jen nenulové řádky,
- pokud rozšiřujeme správně, pak kódování ve 2. doplňku funguje při násobení zcela automaticky,
- výsledek násobení může mít tolik řádů, kolik je součet počtu řádů obou členů.

Příklad:

$$(3)_{10} \cdot (3)_{10} = (9)_{10}$$
$$(11)_2 \cdot (11)_2 = (1001)_2$$

Oba členy mají 2 místa, výsledek má 4.

Násobení ve 2. doplňku - podsebou, kladné členy

Příklad: $6 \cdot 9 = 54$

1. člen: $(110)_2$, po zavedení znaménka $(0110)_2$

2. člen: $(1001)_2$, po zavedení znaménka $(01001)_2$

⇒ oba členy rozšíříme nejméně na 9 bitů - zde 10 bitů

← 0	0	0	0	0	0	0	1	0	0	1
← 0	0	0	0	0	0	0	0	1	1	0
← 0	0	0	0	0	0	0	0	0	0	0
← 0	0	0	0	0	0	1	0	0	1	
← 0	0	0	0	0	1	0	0	1		
← 0	0	0	0	0	1	1	0	1	1	0

Násobení ve 2. doplňku - podsebou, záporný člen

Příklad: $6 \cdot (-9) = -54$

1. člen: $(110)_2$, po zavedení znaménka $(0110)_2$

2. člen: $(10111)_2$

⇒ oba členy rozšíříme nejméně na 9 bitů - zde 10 bitů

← 1	1	1	1	1	1	1	0	1	1	1
← 0	0	0	0	0	0	0	0	1	1	0
← 0	0	0	0	0	0	0	0	0	0	0
← 1	1	1	1	1	1	0	1	1	1	
← 1	1	1	1	1	0	1	1	1		
← 1	1	1	1	1	0	0	1	0	1	0

Násobení ve 2. doplňku

- Dělení v ostatních soustavách funguje podobně jako v desítkové soustavě,
- pro záporná čísla lze používat **metodu dělení s obnovou zbytku** nebo **převedení na absolutní hodnoty**,
- existuje zde problém přetečení při dělení minima rozsahu hodnotou -1:

Příklad:

8 bitový registr s hodnotou se znaménkem ve 2. doplňku.

- Rozsah hodnot -128 až 127,
- problematická operace je $-128 / -1 = 128$, což je mimo rozsah.

⇒ Musím rozšířit dělenec o 1 bit! Výsledek dělení je také širší o 1 bit.

Dělení ve 2. doplňku - kladné členy

Příklad: $15 \cdot 3 = 5$

1. člen: $(1111)_2$, po zavedení znaménka $(01111)_2$

2. člen: $(11)_2$, po zavedení znaménka $(011)_2$, záporná hodnota $(11101)_2$

zbytek		001111	: 11 = 000101
posun dělitele a odečtení od zbytku	1110	100000	+ 001111
záporný výsledek	1110	101111	
posun dělitele a odečtení od zbytku	1111	010000	+ 001111
záporný výsledek	1111	011111	
posun dělitele a odečtení od zbytku	1111	101000	+ 001111
záporný výsledek	1111	110111	
posun dělitele a odečtení od zbytku	1111	110100	+ 001111
kladný výsledek, zbytek	0000	000011	
posun dělitele a odečtení od zbytku	1111	111010	+ 000011
záporný výsledek	1111	111101	
posun dělitele a odečtení od zbytku	1111	111101	+ 000011
kladný výsledek, zbytek	0000	000000	

Téma

1 Sčítání a odčítání v jiných soustavách

2 Vyjádření znaménka - záporná čísla

3 Násobení a dělení

4 Plovoucí řádová čárka

Pevná řádová čárka X Plovoucí řádová čárka

- Pevná řádová čárka má velmi omezený rozsah čísel
Příklad (16 bit) $\implies$ jen 65536 zobrazitelných hodnot, tedy maximální rozsah hodnot je pro:

- celá čísla bez znaménka: 0 až 65536,
- celá čísla se znaménkem: –32768 až 32767,

Po zavedení neceločíselné části se sníží jak jednotka mřížky, tak maximální rozsah hodnot.

- Rozsah lze zvětšit vědeckým zápisem čísel (vědecká notace)

$$m \cdot Z^e$$

Zápis

Ve všech soustavách stejný:

$$m \cdot Z^e$$

- m ... **MANTISA**, informace o číselné hodnotě
- e ... **EXPONENT**, informace o pozici řádové čárky
- Z ... **ZÁKLAD**, základ soustavy vyjadřovaného čísla

Mantisa i exponent mohou nabývat kladných i záporných hodnot.

Příklad v dekadické soustavě pro 2 číslice mantisy a 1 číslici exponentu obojí s přímým zápisem znaménka:

- rozsah hodnot: $-9,9 \cdot 10^9$ až $9,9 \cdot 10^9$
- jednotka, krok: $0,1 \cdot 10^{-9}$

Vlastnosti

Vyjádřením čísel v plovoucí řádové čárce

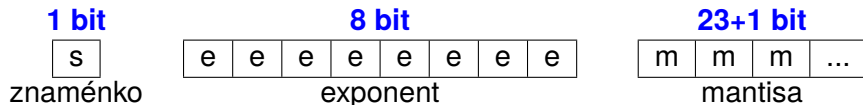
- se nemění počet zobrazitelných hodnot. Pro 16 bitů bude stále počet zobrazitelných hodnot roven 65536,
- lze získat větší rozsah hodnot z pohledu maxima a minima,
- je přesnost každého čísla dána jednotkou zápisu mantisy,
- je třeba vyřešit problém nejednoznačnosti vyjádření téhož čísla, příklad pro dekadickou hodnotu:

$$62,15 = 62,15 \cdot 10^0 = 621,5 \cdot 10^{-1} = 6,215 \cdot 10^1 = \dots$$

- musíme upravit aritmetické operace tak, aby respektovaly přítomnost exponentu, viz dále.

ANSI/IEEE 754 - kódování hodnoty

Jednoduchá přesnost 32 bit:



- Znaménko náleží mantise - přímý kód:
 - 0 ... kladné
 - 1 ... záporné
- Exponent je zapisován aditivním kódem s posunutím **+127**.
- Mantisa je zapisována v normalizovaném tvaru, který obsahuje **skrytou 1**. Mantisa je tak o jeden bit delší než kolik je počet bitů vyhrazených pro její zápis.

ANSI/IEEE 754 - převod

Příklad: $(12, 25)_{10}$

0		1	0	0	0	1	0	0	0	1,	1	0	0	0	1	0	...
---	--	---	---	---	---	---	---	---	---	----	---	---	---	---	---	---	-----

- Převod: $(1100, 01)_2$
- **Znaménko:** kladné $\Rightarrow 0$
- Normalizace - posun řádové čárky po za první zapsanou jedničku
 $(1100, 01)_2 = (1, 10001)_2 \cdot 2^3$
- **Mantisa:** pouze hodnota bez první jedničky $\Rightarrow (10001)_2$
- **Exponent:** $127 + 3 = 130 = (10001000)_2$

ANSI/IEEE 754 - vyhrazené hodnoty

- **Malé hodnoty** v intervalu $-1 \cdot 2^{-126} < x < +1 \cdot 2^{-126}$
 - mají exponent roven 0
 - se nenormalizují - není to možné
- **Nula**
 - mantisa a zároveň exponent jsou nulové
 - může být záporná
- **Nekonečno**
 - exponent je 255 a zároveň **je** mantisa nulová
 - může být kladné i záporné
- **NaN** - chybná hodnota
 - exponent je 255 a zároveň mantisa **není** nulová

ANSI/IEEE 754 - přesnost

Název	Délka	Znaménko	Exponent	Mantisa
poloviční	16	1	5	10+1
jednoduchá	32	1	8	23+1
dvojitá	64	1	11	52+1
čtyřnásobná	128	1	15	112+1

Aritmetika

1 Aritmetická operace

• sčítání (odečítání)

- 1 Úprava zápisu obou hodnot tak, aby měly shodné exponenty.
- 2 Sečtení (odečtení) mantis v přímém kódu.

• násobení

- 1 Součet exponentů v aditivním kódu.
- 2 Násobení mantis.
- 3 Násobení znamének - jde vlastně o sečtení znamének, funkce XOR.

• dělení

- 1 Rozdíl exponentů v aditivním kódu.
- 2 Dělení mantis.
- 3 Totéž co pro násobení znamének.

2 Normalizace výsledku

Následující přednáška

- Booleova algebra
- Logická funkce a její minimalizace
- Hradla a kombinační logické obvody

Děkuji za pozornost